

Task 2: Reflection on My Recent AI-Related Experiences

During the past ten days, the AI-related events that influenced me most were the lectures on agentic AI design patterns, my implementation of the receipt-processing homework, and my experience of using Codex as a programming assistant. Together, these experiences changed my view of AI from a single model that produces an answer into a complete system that must be designed, tested, monitored, and corrected.

Before these experiences, I tended to evaluate an AI solution mainly by the capability of its underlying model. My previous projects in computer vision and time-series forecasting also reinforced this model-centred perspective: I usually focused on improving architectures, accuracy, and computational efficiency. The introductory lecture challenged this assumption by describing the shift from *models* to *systems*. An LLM may be the cognitive engine, but reliable applications also require control flow, tools, state, validation, recovery mechanisms, and sometimes human approval. This made me realise that a strong model is only one component of a useful AI product.

The homework made this lesson concrete. I built a two-stage workflow in which the first prompt extracts `paid`, `subtotal`, and `discounts` from each receipt, while a second prompt audits the first result against the original image. This directly reflects the prompt-chaining pattern discussed in class: a complex task is divided into focused stages with a clear structured interface. JSON output, `Decimal` arithmetic, validation rules, and fallback logic serve as deterministic checks between model calls. Because different receipts are independent, I also used parallel batch processing and then converged the results into two totals. This corresponds to the parallelisation principle of running independent work concurrently before performing sequential synthesis. Finally, the program routes invalid outputs through alternative paths - audited output, first-pass output, and one retry - which showed me that routing is not only about classifying user intent; it can also support error recovery based on intermediate state.

One failure was particularly valuable. A single receipt initially worked, but five receipts failed during the complete public test because the model responses contained no final JSON. The problem was not simply that the vision model was inaccurate. The output limit was too small for the model's reasoning process, so more complex responses were truncated before reaching the required structured answer. After removing that restrictive limit, all seven receipts were processed correctly. This incident changed how I understand reliability: end-to-end failure can come from an interface, token budget, parser, concurrency setting, network configuration, or recovery policy even when the model itself is capable. The lecture's warning that more moving parts create more possible breakdowns became very real to me.

Using Codex also changed my perspective on human-AI collaboration. It helped me inspect code, formulate prompts, diagnose dependency and proxy problems, and review Git changes. However, it was not automatically correct. At one point, an AI recommendation about making the repository private conflicted with the assignment PDF, which explicitly required a public repository. Re-reading the primary source corrected the mistake. This experience connects closely to the lecture's discussion of AI and learning: AI is most valuable to me as a tutor and thinking partner, not as a tool for outsourcing judgment. I should be able to explain the generated code, test it independently, and verify important claims against authoritative evidence.

These experiences have broadened my career plan. I am still interested in model research, especially computer vision, efficient architectures, and time-series forecasting, but I no longer want to focus only on improving benchmark accuracy. I now want to develop toward an AI systems or agentic AI engineer who can connect models with real workflows and make them reliable enough for industrial and financial applications. In these domains, auditability, cost, latency, privacy, human escalation, and failure recovery can be as important as raw model performance.

My next steps are therefore more concrete. I plan to deepen my knowledge of LangChain and LangGraph, learn tool calling, state and memory management, and practise choosing between chaining, routing, and parallelisation instead of applying every pattern unnecessarily. I also want to evaluate future projects with system-level measures such as end-to-end accuracy, latency, cost, retry rate, and failure rate. Most importantly, I will continue using AI assistants, but I will treat their outputs as hypotheses to be checked rather than conclusions to be copied. The most important change in my perspective is that building useful AI is not only about making a model more intelligent; it is about designing a trustworthy process around that intelligence.



AI-use note: I used an AI assistant to help organise and edit this reflection. The experiences, interpretations, and career decisions described here are my own.